KIẾN THỨC VỀ OPENSTACK

I. Keystone

1. Keystone (Dịch vụ API)

- Keystone cấp phát Fernet tokens (không yêu cầu lưu trữ trong cơ sở dữ liệu), và dựa vào các khóa mã hóa đối xứng AES256 (lưu trữ các khóa mã hóa AES256 keys tại /etc/keystone/fernet-keys) để xác thực dữ liệu

```
root@openstack-aio:/etc/kolla# docker exec keystone ls -l /etc/keystone/fernet-keys
total 12
-rw------- 1 keystone keystone 44 Mar  8 00:00 0
-rw------- 1 keystone keystone 44 Feb 28 12:59 1
-rw------- 1 keystone keystone 44 Feb 28 12:59 2
```

- Tệp số cao nhất (ví dụ: 2): Là Primary Key, dùng để mã hóa và tạo ra Token mới.
- Các tệp số thấp hơn (ví dụ: 1): Là Secondary Keys, chỉ dùng để giải mã các Token cũ còn hạn.
- Tệp số 0: Thường là Staged Key, sẽ trở thành Primary Key trong lần xoay vòng (rotation) tiếp theo.

2. keystone_fernet (Trình xoay vòng khóa)

- Thực thi các tác vụ xoay vòng khóa, lệnh keystone-manage fernet_rotate. Lệnh này sẽ tạo khóa mới, đẩy khóa cũ thành Secondary và xóa khóa quá hạn.

3. keystone_ssh

- Một container máy chủ SSH được sử dụng độc quyền làm lớp vận chuyển để đồng bộ hóa các khóa Fernet
- Kích hoạt cơ chế đồng bộ hóa (rsync) để tất cả các controller khác nhận được các khóa đã cập nhật.

4. Luồng hoạt động

1) **Khởi tạo:** Keystone tạo cặp khóa AES256 trong /etc/keystone/fernet-keys/.
2) **Cấp phát:** Người dùng đăng nhập → Keystone dùng Primary Key mã hóa thông tin → Sinh ra chuỗi Fernet Token gửi cho người dùng.
3) **Xác thực:** Người dùng gửi Token kèm request → Keystone dùng các khóa local (0, 1, 2...) để giải mã. Nếu khớp → Cho phép truy cập.
4) **Xoay vòng:** Định kỳ, cronjob hoặc hệ thống quản lý chạy lệnh rotate → Khóa mới được sinh ra → Keystone_ssh đồng bộ tệp khóa này khắp cụm Cluster để đảm bảo tính nhất quán (Consistency).

II. Nova

1. **nova_api**

- Tiếp nhận các yêu cầu REST API từ người dùng hoặc service khác, thực hiện xác thực qua Keystone và đưa yêu cầu vào hàng đợi (Message Queue).

2. **nova_scheduler**

- Nó nhận yêu cầu tạo máy ảo từ message queue (hàng đợi tin nhắn) và quyết định xem máy ảo nên chạy trên compute server host nào.
- Áp dụng trọng số (weights) để chọn host tối ưu nhất

3. **nova_conductor**

- Làm trung gian tương tác giữa dịch vụ nova-compute và database, loại bỏ việc các compute node truy cập trực tiếp vào cloud database để giảm thiểu rủi ro bảo mật

4. **nova_metadata**

- Khi máy ảo khởi động, nó sẽ gửi request đến địa chỉ IP ảo 169.254.169.254. Dịch vụ này sẽ trả về các thông tin như: tên máy ảo, SSH Key, script khởi tạo (cloud-init).

5. **nova_compute**

- Quản lý các máy ảo bằng cách tạo và chấm dứt các instance thông qua các API của hypervisor.
- Liên tục lắng nghe message queue, tiếp nhận các hành động và thực thi các lệnh hệ thống để cập nhật trạng thái của instance trong database.

6. **nova_libvirt**

- nova_compute không trực tiếp ra lệnh cho phần cứng. Nó gửi lệnh đến nova_libvirt. Libvirt sẽ dịch các lệnh đó thành tệp cấu hình XML và khởi chạy tiến trình máy ảo thực sự thông qua KVM.
- Libvirt là driver ảo hóa được sử dụng phổ biến nhất trong OpenStack, thường được hỗ trợ bởi QEMU và KVM để cung cấp ảo hóa tăng tốc phần cứng.

7. **nova_ssh**

- Quan trọng đối với các hoạt động như cold migration (di chuyển lạnh), resize (thay đổi kích thước) và live migration (di chuyển trực tiếp), nơi trình nền libvirtd (chạy dưới quyền root) sử dụng giao thức SSH để sao chép bộ nhớ và nội dung ổ đĩa của máy ảo sang compute host đích

8. **nova_novncproxy**

- Cung cấp giao diện màn hình máy ảo qua trình duyệt web.
- Ví dụ: Proxy này sẽ đứng giữa, chuyển đổi dữ liệu VNC thành định dạng WebSockets để bạn có thể thấy màn hình máy ảo ngay trên giao diện Horizon.

III. Neutron

1. **neutron_server**

- Tiếp nhận và định tuyến các yêu cầu API từ người dùng hoặc các dịch vụ khác (như Nova)
- Khi sử dụng OVN, plugin ML2/OVN được tải bên trong để chuyển đổi các đối tượng mạng của OpenStack thành dữ liệu trong OVN Northbound Database (NB DB).
- Khi tạo một network hoặc một port, dịch vụ này nhận yêu cầu HTTP, xác thực nó và ghi trạng thái vào cơ sở dữ liệu SQL của Neutron

2. **neutron_rpc_server**

- Mặc dù OVN sử dụng Database để giao tiếp, OpenStack vẫn có một số Agent hoặc thành phần cũ (legacy) cần nói chuyện với Controller qua Message Queue (RabbitMQ).
- Do đó, neutron_rpc_server được khởi chạy như một tiến trình độc lập để lắng nghe hàng đợi tin nhắn (RabbitMQ).

3. **neutron_ovn_metadata_agent**

- Làm proxy chuyển tiếp các yêu cầu metadata từ máy ảo đến Nova Metadata API để lấy thông tin (như SSH Key) và trả về cho máy ảo.

4. **neutron_ovn_maintenance_worker**

- Đảm bảo tính nhất quán giữa cơ sở dữ liệu SQL của Neutron và các cơ sở dữ liệu của OVN.
- Worker này đặc thù dành cho driver ML2/OVN. Công việc của nó là chạy các bài kiểm tra, nếu nó phát hiện một mạng tồn tại trong cơ sở dữ liệu Neutron nhưng lại bị thiếu trong OVN Northbound (có thể do gián đoạn mạng tạm thời), worker này sẽ tự động sửa chữa trạng thái bằng cách đẩy dữ liệu còn thiếu sang OVN.

5. **neutron_periodic_worker**

- Một trình nền (daemon) chuyên dụng để chạy các tác vụ mạng định kỳ (ví dụ: dọn dẹp các bản ghi cơ sở dữ liệu cũ), theo lịch trình.
- Khác với Maintenance Worker (tập trung vào OVN), Periodic Worker tập trung vào các logic nghiệp vụ chung của Neutron.

IV. Placement

- Placement chỉ đơn giản là theo dõi kho tài nguyên và trả về danh sách các ứng viên có đủ dung lượng và các đặc điểm phù hợp
- nova-scheduler vẫn chịu trách nhiệm đưa ra quyết định cuối cùng (áp dụng các bộ lọc và trọng số như affinity/anti-affinity hoặc phân bổ mức sử dụng RAM)
- Ví dụ:

1. Yêu cầu tạo VM, Nova API gửi yêu cầu đến Nova Scheduler.

2. Nova Scheduler không tự quét các node mà sẽ hỏi Placement API: "Danh sách các Resource Providers nào có đủ 4 CPU và 8GB RAM còn trống?".
3. Placement API lọc nhanh trong cơ sở dữ liệu và trả về danh sách các ứng viên đủ điều kiện.
4. Nova Scheduler chỉ việc chấm điểm (weighting) trên danh sách rút gọn này để chọn node cuối cùng.

V. OVN

1. ovn_nb_db (Northbound Database)

- Khi tạo một network, router hoặc security group trong OpenStack, plugin Neutron ML2/OVN sẽ ghi dữ liệu này trực tiếp vào Northbound DB.
- Nó lưu trữ các đối tượng thuần túy logic như Logical_Switch, Logical_Router, ACL, và Address_Set.
- Nó hoàn toàn không biết gì về các compute node hay hypervisor vật lý.

2. ovn_sb_db (Southbound Database)

- Cơ sở dữ liệu này liên kết mạng logic với hạ tầng vật lý.
- Nó duy trì các bảng như Chassis (để biết compute node nào là node nào), Port_Binding (port nào nằm trên host nào), và các luồng logic chi tiết do ovn_northd tạo ra

3. ovn_northd

- Một dịch vụ tập trung đóng vai trò là trình biên dịch giữa cơ sở dữ liệu Northbound và Southbound.
- Nó lắng nghe các ý định logic trong ovn_nb_db và biên dịch chúng thành các luồng logic (logical flows), liên kết đường dẫn dữ liệu (datapath bindings) và các nhóm multicast, sau đó ghi chúng vào Southbound DB.

4. ovn_sb_db_relay_1

- Một bản sao chỉ đọc (read-only replica) của Southbound DB dùng để mở rộng quy mô
- Kolla-Ansible tự động cấu hình các nhóm relay (ví dụ: tạo 1 relay cho mỗi 50 compute host theo mặc định) để phân tán tải đọc cơ sở dữ liệu . Các compute node sẽ kết nối với relay thay vì database master trung tâm.

5. ovn_controller

- Bộ điều khiển mạng phân tán thay thế cho Neutron OVS layer-2 agent truyền thống.
- Nó chạy trên mọi hypervisor và kết nối với Southbound DB (hoặc relay của nó). Nó đăng ký host của mình dưới dạng một Chassis. Nó đọc các luồng logic được gán cho chassis cụ thể của nó và biên dịch chúng thành các quy tắc OpenFlow vật lý, đẩy chúng xuống OVS integration bridge cục bộ (br-int). Nó cũng xử lý việc phản hồi ARP cục bộ và giám sát BFD.

VI. Open vSwitch

1. openvswitch_db (OVSDB Server)

- Dịch vụ cơ sở dữ liệu cục bộ cho Open vSwitch.
- dịch vụ này chứa cấu hình OVS cục bộ (conf.db) trên hypervisor. Nó theo dõi các interface vật lý, các bridge (như br-int và br-ex), và các thống kê port cho máy chủ cụ thể đó.

2. openvswitch_vswitchd (OVS Data Plane)

- Đây là trình nền (daemon) cốt lõi thực sự xử lý các gói tin mạng.
- Nó thực thi các quy tắc OpenFlow do ovn_controller lập trình, thực hiện toàn bộ việc chuyển mạch (switching), định tuyến HA (HA routing), và kiểm soát truy cập (Security Groups) trực tiếp trong kernel hoặc user-space datapath.

3. Ví dụ: Tạo một máy ảo (VM) mới trên Compute Node A

1. Neutron nhận API request và tạo một logical port trong **ovn_nb_db**.
2. **ovn_northd** nhìn thấy port mới này, tạo ra các luồng logic tương ứng và ghi một Port_Binding vào **ovn_sb_db**.
3. **ovn_controller** đang chạy trên Compute Node A phát hiện ra rằng port này được lên lịch cho chassis của nó (thông qua **ovn_sb_db_relay_1**).
4. ovn_controller biên dịch các luồng logic thành các quy tắc OpenFlow và ghi chúng vào **openvswitch_vswitchd** cục bộ thông qua **openvswitch_db** cục bộ.
5. Lưu lượng mạng của VM hiện đã được định tuyến và chuyển mạch an toàn cục bộ trên host.

VII. Cinder

1. cinder_api

- Tiếp nhận các yêu cầu HTTP REST từ người dùng hoặc các dịch vụ khác (như Nova).
- Nó thực hiện xác thực thông qua Keystone, kiểm tra hạn mức (Quotas) và đẩy tin nhắn vào hàng đợi (Message Queue) để các thành phần khác xử lý.

2. cinder_scheduler

- Dịch vụ chịu trách nhiệm xác định chính xác storage node (node lưu trữ) nào sẽ thực hiện yêu cầu tạo volume

3. cinder_volume

- Trình nền (worker daemon) quản lý các thiết bị lưu trữ vật lý hoặc các hệ thống lưu trữ định nghĩa bằng phần mềm (Software-Defined Storage)
- Dịch vụ này chạy trên các storage node và lắng nghe các lệnh từ hàng đợi tin nhắn.
- Nó dựa vào một kiến trúc driver (trình điều khiển) có thể cắm (pluggable) để biên dịch các lệnh OpenStack thành các hành động đặc thù của từng hãng lưu trữ. Bất kể backend của là LVM cục bộ, Ceph RBD, NetApp, hay Dell PowerMax, cinder_volume sẽ sử dụng driver tương ứng của nhà cung cấp để

cấp phát không gian, tạo snapshot và export (xuất) các LUN (Logical Unit Number) để các node Compute có thể mount chúng thông qua iSCSI, Fibre Channel, hoặc NVMe-oF.

4. **cinder_backup**

- Một dịch vụ chuyên dụng để sao lưu (backup) các Cinder volume ra một kho lưu trữ bên ngoài.
- Dịch vụ cinder_backup đọc dữ liệu khối từ một volume và đẩy luồng (stream) dữ liệu đó sang một hệ thống lưu trữ thứ cấp (thường có chi phí rẻ hơn). Mặc định, nó sao lưu sang OpenStack Object Storage (Swift), nhưng nó cũng hỗ trợ các driver cho Ceph, NFS, Google Cloud Storage,..

VIII. Manila

1. **manila_api**

- Nó tiếp nhận các yêu cầu từ người dùng cuối hoặc các dịch vụ OpenStack khác, xác thực chúng thông qua Keystone, thực hiện xác thực yêu cầu cơ bản và định tuyến các yêu cầu xuyên suốt dịch vụ bằng cách đẩy các tin nhắn vào message queue (hàng đợi tin nhắn). Nó không tự mình thực hiện các thao tác lưu trữ.

2. **manila_scheduler**

- Dịch vụ chịu trách nhiệm quyết định chính xác storage backend (hệ thống lưu trữ phía sau) nào sẽ thực hiện yêu cầu tạo share (phân vùng chia sẻ).

3. **manila_share**

- Trình nền (worker daemon) quản lý các thiết bị backend cung cấp hệ thống tệp chia sẻ.
- Dịch vụ này giao tiếp với các thiết bị lưu trữ vật lý hoặc định nghĩa bằng phần mềm thông qua các share back-end drivers đóng vai trò là giao diện. Điểm mấu chốt là một share driver hoạt động ở một trong hai chế độ:
    - Có quản lý share server (driver_handles_share_servers = True): Driver tự động cấp phát các "share servers" (như các máy ảo dịch vụ) và cắm chúng vào các "share networks" (của Neutron) do người dùng định nghĩa để xuất (export) các file shares
    - Không quản lý share server (driver_handles_share_servers = False): Các yêu cầu về mạng được quản trị viên xử lý bên ngoài (out-of-band) và Manila chỉ đơn giản là cấp phát không gian lưu trữ

4. **manila_data**

- Một dịch vụ độc lập được thiết kế đặc biệt để xử lý các hoạt động dữ liệu nặng, mất nhiều thời gian.
- manila_data tiếp nhận các yêu cầu cho các tác vụ như sao chép dữ liệu, thực hiện backup share, hoặc thực thi "host-assisted migrations" (di chuyển một

share từ hệ thống lưu trữ của hãng này sang hãng khác thông qua mạng quản trị)

IX. Glance

- Nó hoạt động như một dịch vụ đăng ký và phân phối cho các tệp ảnh máy ảo (VM images) và các bản sao lưu ổ đĩa (volume snapshots)
- Control Plane: Khi một image được tải lên, glance_api sẽ quản lý metadata (như định dạng đĩa qcow2 hoặc raw, định dạng container, UUID, kích thước và các thẻ tags) và lưu trạng thái này vào cơ sở dữ liệu của OpenStack
- glance_api không thực sự lưu trữ các tệp nhị phân lớn của image bên trong cơ sở dữ liệu của chính nó. Thay vào đó, nó sử dụng thư viện glance_store có thể cắm ghép (pluggable). Nó truyền luồng (stream) dữ liệu nhị phân của image đến và đi từ các hệ thống lưu trữ backend.
- Các backend được hỗ trợ bao gồm hệ thống tệp cục bộ (local file system), OpenStack Object Storage (Swift), Ceph (RADOS Block Devices), VMware datastores, hoặc thậm chí là các máy chủ HTTP từ xa.

X. Infrastructure

1. mariadb

- Các dịch vụ OpenStack (Nova, Cinder, Neutron, Keystone, v.v.) về cơ bản là phi trạng thái (stateless); chúng lưu trữ toàn bộ siêu dữ liệu vĩnh viễn của mình—như trạng thái máy ảo, định nghĩa mạng ảo, và thông tin xác thực người dùng—trong MariaDB.
- Để đảm bảo Tính sẵn sàng cao (HA), MariaDB được triển khai dưới dạng một Galera Cluster, đây là một cụm đa master đồng bộ (synchronous multi-master)

2. proxysql

- ProxySQL đứng giữa ứng dụng và các node MariaDB. Nó có khả năng Read/Write Splitting: Tự động đẩy các câu lệnh INSERT/UPDATE (Ghi) vào node Master và các câu lệnh SELECT (Đọc) vào các node Slave để giảm tải cho Master

3. rabbitmq

- Các dịch vụ OpenStack giao tiếp với nhau một cách bất đồng bộ bằng cách sử dụng Gọi hàm từ xa (RPC) thông qua RabbitMQ. Ví dụ, nova-api không giao tiếp trực tiếp với nova-compute qua HTTP; thay vào đó, nó thả một tin nhắn vào hàng đợi RabbitMQ, và compute worker sẽ nhặt nó lên.
- RabbitMQ quản lý việc xếp hàng, phân phối và phân cụm (clustering), đồng thời đóng vai trò cực kỳ quan trọng đối với tính sẵn sàng cao của OpenStack.
- Các triển khai hiện đại sử dụng "hàng đợi túc số" (quorum queues - dựa trên thuật toán đồng thuận Raft) để đảm bảo tin nhắn được sao chép và luôn sẵn sàng trên toàn cụm RabbitMQ.

4. **memcached**

- Một hệ thống lưu trữ key-value trên RAM phân tán, hiệu năng cao
- Nhiệm vụ quan trọng nhất của nó trong OpenStack là lưu trữ đệm các token xác thực của Identity (Keystone). Vì mọi tương tác dịch vụ đều yêu cầu xác thực token, việc cache chúng trong memcached giúp ngăn chặn việc đọc database quá mức.
- Nó cũng được giao diện Horizon sử dụng để lưu trữ session và được Swift dùng để cache sự tồn tại của các container/account.

XI. Aodh

1. **aodh_api**

- Tiếp nhận các yêu cầu để tạo, cập nhật, xóa hoặc xem danh sách các quy tắc cảnh báo (alarm rules).
- Dữ liệu cấu hình cảnh báo sẽ được lưu vào cơ sở dữ liệu (MariaDB).

2. **aodh_evaluator**

- Chạy định kỳ, thực hiện truy vấn dữ liệu từ các dịch vụ lưu trữ chỉ số (thường là Gnocchi hoặc Prometheus) để so sánh giá trị thực tế với ngưỡng đã thiết lập trong aodh_api.
- Nếu điều kiện vi phạm, nó sẽ gửi tin nhắn tới aodh_notifier.

3. **aodh_listener**

- Lắng nghe trực tiếp trên hàng đợi tin nhắn (RabbitMQ).
- Khi có một sự kiện cụ thể xảy ra trong hệ thống OpenStack (ví dụ: máy ảo bị tắt đột ngột), listener sẽ bắt lấy và kích hoạt cảnh báo ngay lập tức.

4. **aodh_notifier**

- Trạng thái của một cảnh báo thành "alarm" hoặc "ok", aodh_notifier sẽ tiếp quản
- Nó thực thi hành động đã được xác định, thường là gửi một yêu cầu HTTP/HTTPS POST (webhook) với JSON payload mô tả sự thay đổi trạng thái
- Nó cũng có thể ghi vào file log, gửi tin nhắn vào hàng đợi Zaqar, hoặc kích hoạt các cơ chế tự phục hồi (auto-healing) của Heat

XII. Ceilometer

1. **ceilometer_central**

- Truy vấn (poll) các API của OpenStack để lấy thống kê mức độ sử dụng của các tài nguyên độc lập với các máy ảo hoặc hypervisor cụ thể
- Ví dụ: nó định kỳ hỏi Cinder API về kích thước volume, Glance API về kích thước image, Swift API về dung lượng object storage đã tiêu thụ, và Neutron API về các port mạng.

2. **ceilometer_compute**

- Thay vì truy vấn các API qua mạng, agent này giao tiếp trực tiếp với hypervisor cục bộ (như KVM/libvirt)
- Nó chủ động truy vấn hypervisor để trích xuất các thống kê sử dụng tài nguyên dành riêng cho các máy ảo đang chạy trên chính máy chủ vật lý đó (chẳng hạn như mức sử dụng CPU, RAM, I/O ổ đĩa, và I/O mạng)

3. **ceilometer_notification**

- Các dịch vụ OpenStack (Nova, Cinder, Keystone, v.v.) liên tục phát ra các tin nhắn thông báo (notification messages) vào message bus bất cứ khi nào có sự thay đổi trạng thái (ví dụ: khi một máy ảo được tạo, bị xóa, hoặc một volume được gắn vào)
- Ceilometer_notification thụ động lắng nghe các topic này, bắt lấy các sự kiện, biên dịch chúng thành các metric Telemetry tiêu chuẩn, và đẩy chúng đến dịch vụ collector (hoặc đẩy trực tiếp vào cơ sở dữ liệu chuỗi thời gian như Gnocchi)

XIII. Cron

- cron được sử dụng để kích hoạt các tác vụ bảo trì định kỳ trên toàn bộ cloud.
- Vai trò quan trọng nhất của nó là thực hiện xoay vòng log (log rotation) (ngăn chặn các tệp log làm đầy ổ cứng)
- Kích hoạt việc xoay vòng khóa Keystone Fernet.
- Bằng cách tách các tác vụ này vào một dịch vụ cron chuyên dụng, các API lõi của OpenStack không phải tự quản lý các bộ đếm thời gian chạy ngầm của riêng chúng.

XIV. Fluentd

- **Chạy trên mọi node** để bắt các tệp log (và các sự kiện kiểm toán CADF từ hàng đợi tin nhắn) và chuyển (ship) chúng đến một backend tìm kiếm tập trung, như OpenSearch hoặc Elasticsearch
- Hoạt động như một lớp ghi log thống nhất, lọc và định tuyến log mà không làm nghẽn các dịch vụ OpenStack đang tạo ra chúng.

XV. Gnocchi

1. **gnocchi_api**

- Nhận các chỉ số đo lường (metrics) từ các tác nhân Ceilometer.
- Để đảm bảo tính sẵn sàng cao và thời gian phản hồi cực nhanh, gnocchi_api không xử lý hoặc tổng hợp dữ liệu một cách đồng bộ (synchronous). Thay vào đó, nó chỉ đơn giản là xác thực yêu cầu, cập nhật chỉ mục tài nguyên (lưu trong MariaDB/PostgreSQL), và ghi dữ liệu đo lường thô vào một khu vực lưu trữ đến tạm thời (thường là Ceph hoặc Swift).

2. **gnocchi_metricd**

- Nó liên tục quét khu vực lưu trữ tạm thời để tìm các chỉ số thô mới bị gnocchi_api thả vào. Khi tìm thấy, gnocchi_metricd sẽ tính toán các giá trị tổng

hợp (chẳng hạn như mức sử dụng CPU trung bình, tối thiểu và tối đa trong khoảng thời gian 5 phút) dựa trên các "Chính sách Lưu trữ" (Archive Policies) đã được định nghĩa.

- Sau đó, nó nén và lưu trữ vĩnh viễn dữ liệu chuỗi thời gian đã xử lý này vào backend lưu trữ (như Swift hoặc Ceph).

XVI. tgtd

- Khi sử dụng backend Cinder LVM, tgtd đóng vai trò là cầu nối giữa tiến trình cinder-volume và các Nhóm Volume Logic (LVG) vật lý.
- Nó lấy một logical volume được cắt ra cục bộ và export nó qua mạng bằng giao thức iSCSI, trình bày nó dưới dạng một iSCSI Target với một IQN duy nhất.

XVII. iscsid

- Khi một máy ảo cần đính kèm một volume, iscsid hoạt động như một "initiator" (bộ khởi tạo)
- Nó chủ động tìm kiếm tgtd target trên storage node, xác thực (thường sử dụng CHAP) và đăng nhập. Sau khi kết nối thành công, kernel Linux trên compute node sẽ nhìn thấy không gian lưu trữ từ xa đó như một thiết bị khối (block device) cục bộ (ví dụ: /dev/sdb).

XVIII. multipathd

- Quản lý nhiều đường dẫn phần cứng đến cùng một thiết bị lưu trữ.
- iscsid có thể nhìn thấy chính xác cùng một volume từ xa hai lần (ví dụ: qua IP A và IP B). multipathd chặn các đường dẫn trùng lặp này và gộp chúng lại thành một thiết bị khối ảo duy nhất

XIX. kolla_toolbox

- kolla_toolbox là một môi trường được đóng gói sẵn các module Ansible, thư viện Python và các client cơ sở dữ liệu.
- Kolla-Ansible kết nối tới container này để thực hiện các tác vụ quản trị đặc quyền (như migration cơ sở dữ liệu, chạy các cron job, hoặc thực thi các lệnh OpenStack CLI) mà không làm ô nhiễm hệ điều hành máy chủ

XX. Grafana

- Một nền tảng trực quan hóa dữ liệu và khả năng quan sát mở, có thể lắp ghép.
- Rất tốt trong việc vẽ biểu đồ các chỉ số do Prometheus thu thập. Nó cung cấp các bảng điều khiển (dashboards) được xây dựng sẵn hiển thị các đợt tăng vọt CPU, độ trễ API và tình trạng khóa (locks) cơ sở dữ liệu.

XXI. Prometheus

1. prometheus_server

Hệ thống giám sát và cơ sở dữ liệu chuỗi thời gian cốt lõi. Nó chủ động "scrape" (kéo/truy vấn) các endpoint HTTP trên tất cả các node OpenStack theo các khoảng thời gian đều đặn (ví dụ: mỗi 60 giây) để thu thập các chỉ số hiệu suất

2. prometheus_alertmanager

Xử lý các cảnh báo được gửi bởi máy chủ Prometheus. Nó loại bỏ trùng lặp (deduplicate), nhóm và định tuyến các cảnh báo này đến đúng người nhận (như email, Slack, PagerDuty, hoặc Webhooks)

3. Prometheus Exporters

- prometheus_node_exporter: Chạy trên mọi máy chủ vật lý để phơi bày các chỉ số cấp phần cứng và hệ điều hành (CPU, RAM, Disk I/O, Băng thông mạng).
- prometheus_openstack_exporter: Truy vấn các OpenStack API (Nova, Neutron, Cinder, v.v.) để phơi bày các chỉ số cấp độ cloud (ví dụ: tổng số VM đang chạy, số floating IP đã sử dụng, số volume đang ở trạng thái lỗi).
- prometheus_libvirt_exporter: Chạy trên các compute node để thu thập các chỉ số cấp hypervisor trực tiếp từ KVM/libvirt (ví dụ: mức sử dụng CPU/RAM của các máy ảo cụ thể).
- prometheus_mysqld_exporter: Giám sát sức khỏe, các truy vấn và trạng thái sao chép (replication) của cụm cơ sở dữ liệu MariaDB Galera.
- prometheus_memcached_exporter: Theo dõi tỷ lệ hit/miss của bộ nhớ cache và mức sử dụng bộ nhớ của Memcached (nơi xử lý việc cache token của Keystone).
- prometheus_elasticsearch_exporter: Giám sát sức khỏe và trạng thái shard của chính cụm OpenSearch/Elasticsearch.
- prometheus_cadvisor: Phân tích việc sử dụng tài nguyên và đặc điểm hiệu suất của các container đang chạy (rất hữu ích vì Kolla-Ansible chạy OpenStack bên trong Docker/Podman).
- prometheus_blackbox_exporter: Thăm dò (probe) các endpoint qua HTTP, HTTPS, DNS, TCP và ICMP. Nó hoạt động như một bài kiểm tra tổng hợp từ bên ngoài để xem liệu các OpenStack API (như endpoint công khai của Keystone) có thực sự có thể truy cập được và phản hồi nhanh hay không.

XXII. Opensearch

Lưu trữ khối lượng lớn dữ liệu log và các sự kiện kiểm toán (như CADF) được thu thập và gửi về bởi Fluentd từ tất cả các compute và controller node.

XXIII. HAProxy & Keepalived

- HAProxy container lắng nghe các yêu cầu trên các địa chỉ IP ảo (VIP). HAProxy sẽ nhận yêu cầu và điều hướng (load-balance) nó đến một trong các container dịch vụ đang chạy trên các node Controller khác nhau.
- Keepalived là dịch vụ cung cấp tính năng chuyển đổi dự phòng (failover) thông qua giao thức VRRP (Virtual Router Redundancy Protocol). Các node Controller chạy Keepalived sẽ liên tục gửi tín hiệu "heartbeat" cho nhau. Nếu node Master gặp sự cố, Keepalived trên node Slave sẽ phát hiện và tự động gán địa chỉ VIP sang cho mình.

XXIV. Heat

1. **heat_api**

- Tiếp nhận các yêu cầu tạo, cập nhật hoặc xóa "Stack" (một nhóm tài nguyên như VM, mạng, ổ đĩa) từ người dùng.
- Xác thực người dùng qua Keystone và sau đó chuyển tiếp các yêu cầu đã được phân tích cú pháp đến heat_engine thông qua Message Queue (RabbitMQ).

2. **heat_api_cfn**

- Một lớp API bổ sung cung cấp khả năng tương thích với định dạng Query API của AWS CloudFormation.
- Cho phép người dùng sử dụng các công cụ quản lý của AWS để triển khai tài nguyên trên OpenStack mà không cần sửa đổi nhiều.

3. **heat_engine**

- Lấy các template (HOT - Heat Orchestration Template) từ hàng đợi tin nhắn, phân tích sự phụ thuộc giữa các tài nguyên (ví dụ: phải tạo mạng trước khi tạo máy ảo).
- Sau đó, nó thay mặt người dùng (sử dụng Keystone Trust hoặc Token) để gọi API của các dịch vụ khác như Nova, Neutron, Cinder để hiện thực hóa các tài nguyên đó.